



DATA STRUCTURES

CONTENTS



8. Implementation of Stack ADT

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. In this module we will talk about the implementation of the Stack ADT.

Learning Objectives

The learning objectives of the introductory module are as follows:

- To discuss the different implementations of Stack ADT
- To explain the array based implementation of stack including growable arrays & multiple stacks
- To outline the linked list implementation of Stack
- To discuss the implementation of Stack ADT using ADT list

8.1 Implementation of Stack ADT

Any list implementation such as arrays could be used to implement a stack as was

discussed in the previous module. When using arrays the implementation is **static** and the size of stack is to be fixed and given initially. Another implementation of stacks is linked lists where the stack is dynamic and in general can never become full. We will then explore the use of ADT list to implement stacks.

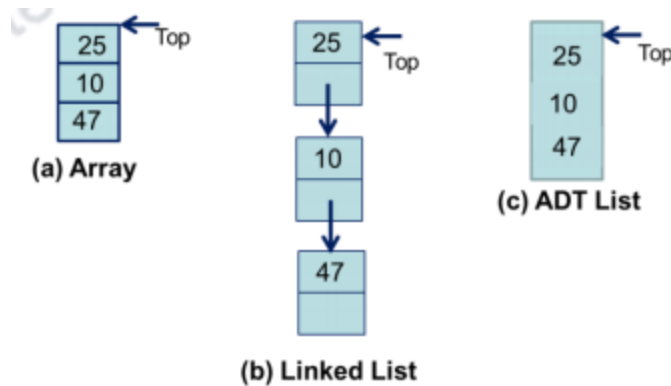


Figure 8.1 Implementation of Stacks

8.2 Array Implementation of Stack ADT

First let us see how to use an array to implement a stack *Stack*. First of all since the implementation is static we need to declare the array size ahead of time. The following are two of the attributes associated with the Stack:

- **MaxSize:** This is the maximum size of the stack and is the size of the array.
- **Top:** This is the index of the top element of stack *Stack* which is the array which stores elements of stack

The following are the operations associated with the stack:

- **IsEmpty:** This returns true if stack is empty, else false
- **IsFull:** This returns true if stack is full, else false. This operation is only defined for Stack ADT when the ADT is represented using an array and hence is implementation dependent.
- **Top:** This returns the element at the top of stack without removing it from the stack
- **Push:** This adds an element to the top of stack
- **Pop:** This deletes the element at the top of stack
- **DisplayStack:** This prints all the data in the stack

Now let us see in detail how the above operations are carried out when the stack is

represented using an array **Stack**. Figure 8.2 shows an array Stack [0,1,.....MaxStack-1] where MaxStack is the maximum size of the stack. The array Stack contains k+1 items of type StackItem Type. The value of Top or index to access the stack in this case is k.

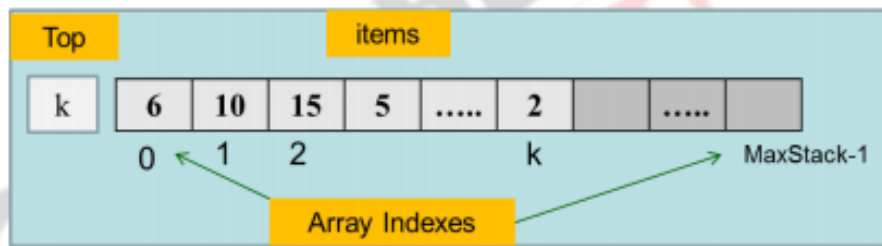


Figure 8.2 An Array-Based Implementation of the ADT Stack

8.2.1 Create Stack

For the creation of the Stack the first step is the allocation of a stack array S of size size = Maxsize. Initially we set Top to -1. This indicates that the stack is empty.

Top $\Rightarrow$ -1

When the stack is full, Top will have its maximum value, i.e. Maxsize - 1

Top $\Rightarrow$ Maxsize -1

8.2.2 Size of the Stack

Initially when the stack is empty, Top has a value = -1. In general we add elements

from left to right (Figure 8.3). Size of the stack is one more than the value of Top

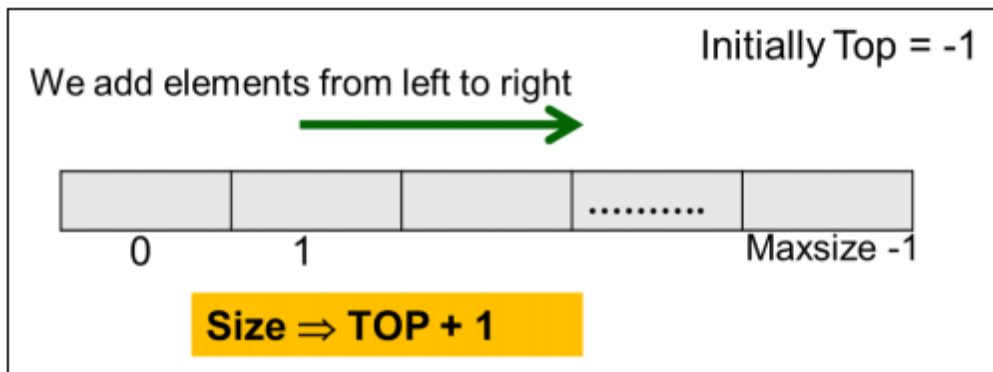


Figure 8.3 Size of the Stack

8.2.3 Push Stack

As we have already discussed, this operation will Push an element onto the stack by incrementing Top. If the stack is full, it will print the error information. Now let us consider an example shown in Figure 8.4.

The steps to be followed are as follows:

Step 1: The array storing the stack elements may become full. If Stack is full that is if **isFull** is true that is **Top = Maxsize - 1** then we signal an error and give an error that the stack is Full or signal a **FullStack Exception**. This is a limitation of the array-based implementation since the stack full condition has to be checked explicitly.

Step 2: Now if isFull is false then we set **Top = Top + 1**. In our example the Top now becomes (2+1) that is 3.

Step 3: Now we set **S[Top]** to be the item to be inserted. In our example 77 is pushed and the value at current Top of the stack is 77.

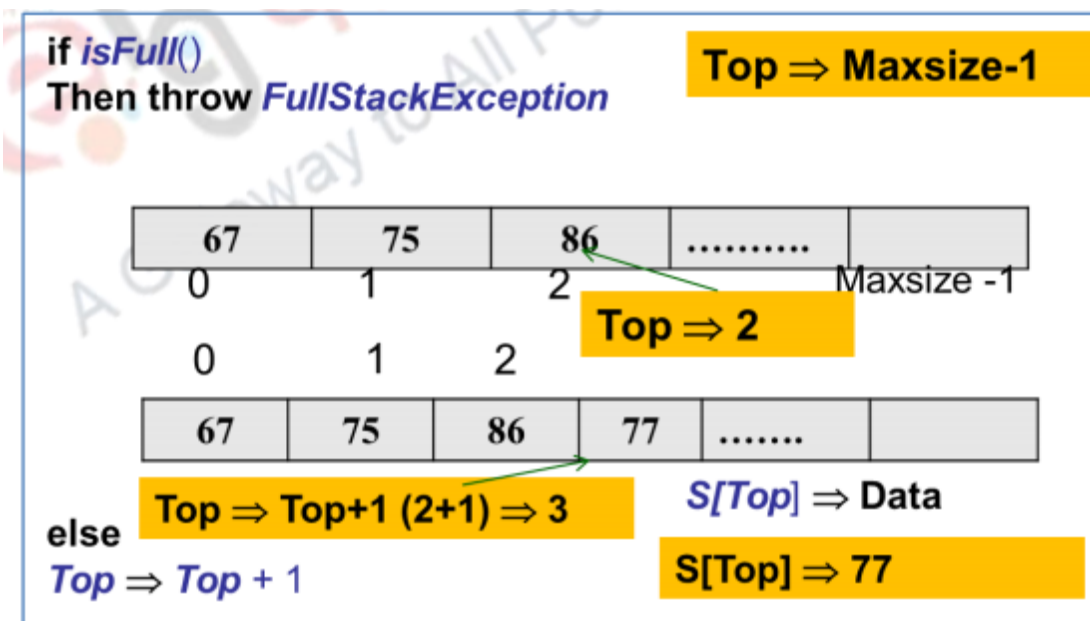


Figure 8.4 Push operation

8.2.4 Pop Stack

As we have already discussed, this operation will Pop and return the element at the top of the stack. If the stack is empty, it will print the error information. (In this case, the return value is useless). In case the stack is not empty we return the value at the top of the stack and decrement Top. Now let us consider an example shown in Figure 8.5.

The steps to be followed are as follows:

Step 1: If Stack is empty that is if *isEmpty* is true that is **Top = -1** then we signal an error and indicate *EmptyStackException*

Step 2: Now if *isEmpty* is false then we set **Top = Top - 1**. In our example the Top now becomes (2-1) that is 1.

Step 3: return **S[Top+1]** . In our example 86 is returned and the value at current Top of the stack is 75. Here the current Top is pointing to the element 75. However we are returning the element at the location **Top+1** which is pointing at the element 86 just deleted from the stack.

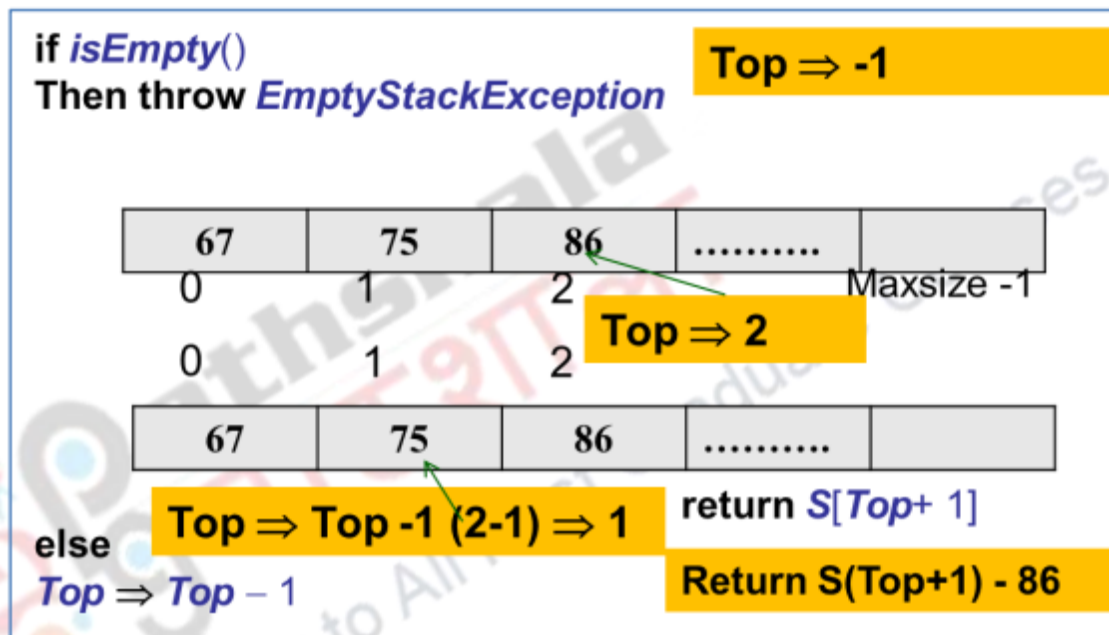


Figure 8.5 Pop operation

8.2.5 Stack Top

This operation just returns the top element of the stack. Unlike Pop, this function does not remove the top element that is the stack remains unchanged.

8.3 Performance and Limitations

Let us now see the performance of the Stack ADT implemented using arrays. Let n be the number of elements in the stack then the space used for storing the elements is $O(n)$. Each operation runs in time $O(1)$ and is therefore efficient. As already discussed one limitation is that the maximum size of the stack must be defined a priori and cannot be changed. Trying to push a new element into a full stack causes an implementation-specific exception.

8.4 Growable Array

The case where the array is full is not an exception defined in the Abstract Stack. If the array is filled then we have five options. The first option is to throw an exception. Another way is to ignore the element that is to be pushed or replace the item at the current top of the stack, Yet another method is to put the pushing

process to “sleep” until something else creates a vacancy. However If dynamic memory is available, the best option is to increase the array capacity.

8.4.1 Array Capacity

Now let us consider the case of dynamic memory being available. The problem is that arrays cannot be resized. You can only copy over elements to a new array. In other words, any time we push onto a full stack there is a requirement of n copies and the run time is $O(n)$. In other words, push is usually $O(1)$ except when new memory is required.

In a push operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one. An important question in that case is how large should the new array be. How much do we increase the capacity – incremental strategy: increase the size by a constant c that is $\text{array_capacity} = \text{original capacity} + c$; or by a multiple that is $\text{array_capacity} = \text{original capacity} * c$. Generally a doubling strategy is common where we double the size of the array.

Figure 8.6 shows an example where doubling strategy has been used. During the push operation when the array S (size 5) storing the stack elements becomes full that is if **isFull** is true that is **Top=Maxsize -1**, instead of signaling a stack full condition, a new array A of size 10 (double original size) is created. Now we need to copy the elements from original array S to the new array A . This is done as follows:

For $i \leftarrow 0$ to Top do $A[i] \leftarrow S[i]$ and now we reassign reference S to the new array A . Then we set Top to $\text{Top}+1$ and insert the new element to S which is now a new array. This array replacement strategy is known as **growable array**, since the process can be seen as growing or extending the end of the underlying array in order to accomodate more elements.

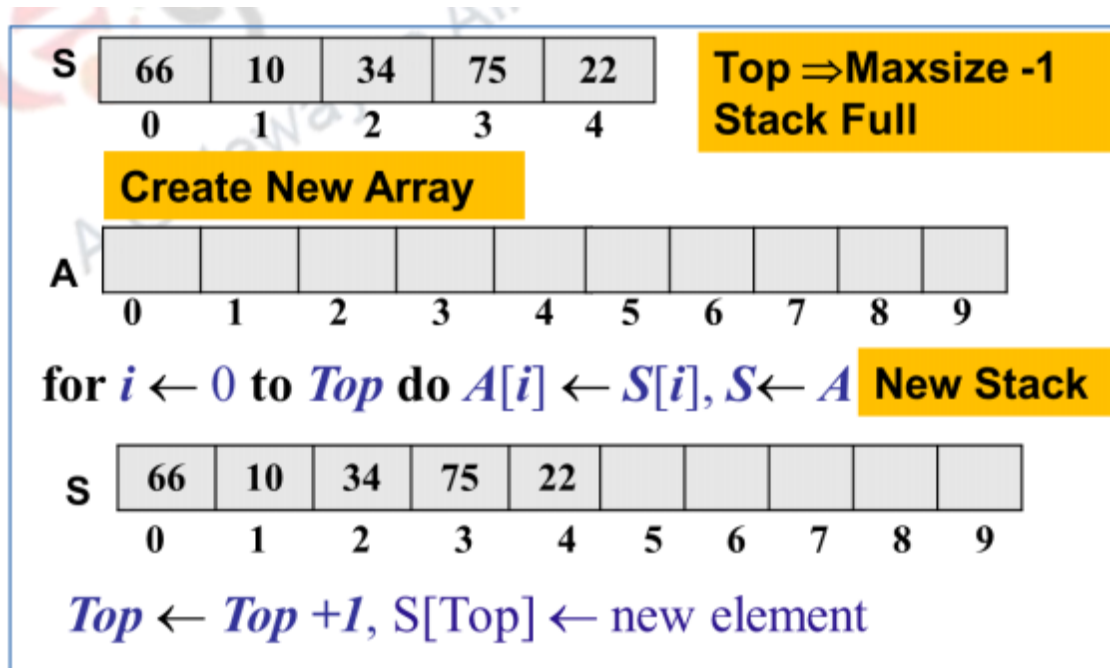


Figure 8.6 Use of Doubling Strategy

8.4.2 Comparison of the Strategies

When we view the incremental strategy from a single operation viewpoint it may seem time consuming since we need to copy all the elements of the original array into the newly created array when a stack overflow occurs. Therefore we need to compare these growable array strategies we analyze the total time $T(n)$ needed to perform a series of n push operations. Let us assume initially that the empty stack is represented by an array of size 1. The amortized time of a push operation which is the average time taken by a push over the series of operations is $T(n)/n$. To define the average run time, we will introduce the concept of amortized time. If n operations requires $O(f(n))$, we will say that an individual operation has an amortized run time of $O(f(n)/n)$. Therefore, if inserting n objects requires: $O(n^2)$ copies, the amortized time is $O(n)$ and for n copies, the amortized time is $O(1)$. Let us see how we carry out amortized for the incremental and doubling strategy.

8.4.2.1 Incremental Strategy Analysis

In the case of incremental strategy let us assume we replace the array $k = n/c$ times. Then The total time $T(n)$ of a series of n push operations is proportional to

$$T(n) = n + c + 2c + 3c + 4c + \dots + kc$$

(the first term n corresponds to n push operations without expansion, the other terms correspond to n/c or k expansions, where each term needs an additional c operations)

Rearranging the terms we get

$$T(n) = n + c(1 + 2 + 3 + \dots + k) - \text{factoring out } c = n + ck(k+1)/2 - \text{rewriting } 1+2+\dots+k \text{ as } k(k+1)/2$$

Since c is a constant, $T(n)$ is $O(n + k^2) = O(n + (n/c)^2)$ that is $O(n^2)$.

Therefore the amortized time of 1 push operation is $O(n)$.

8.4.2.2 Doubling Strategy Analysis

In the case of doubling strategy let us assume we replace the array $k = \log_2 n$ times. Then the total time $T(n)$ of a series of n push operations (Figure 8.7) is proportional to

$$T(n) = n + 1 + 2 + 4 + 8 + \dots + 2^k$$

(the first term n corresponds to n push operations without expansion and the other terms correspond to again k expansions)

Rearranging the terms we get

$$T(n) = n + 2 \times (2^{k+1} - 1) = 3n - 2 = O(n) \text{ (geometric series reduces to } n \text{ - refer to Figure)}$$

$$T(n) \text{ is } O(n)$$

Therefore the amortized time of a push operation is $O(1)$

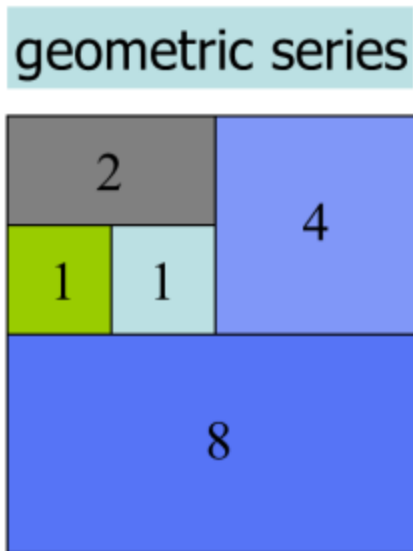


Figure 8.7 Geometric Series

Note the difference in worst-case amortized scenarios for various cases (Figure 8.8)

	Copies per Insertion	Unused Memory
Increase by 1	$n - 1$	0
Increase by m	n/m	$m - 1$
Increase by a factor of 2	1	n
Increase by a factor of $r > 1$	$1/(r - 1)$	$(r - 1)n$

http://www.ece.uwaterloo.ca/~ece250/Algorithms/Array_resizing/

Figure 8.8 Amortized Times for various Strategies

8.5 Multiple Stacks

We can represent two stacks using a single array m (Figure 8.9). When we need more than two stacks, we can:

- Use a fixed partition for each stack.
- Use a variable partition for each stack. When stack is full

1. Find a larger, free space.
2. Move the related stacks around.

When an array represents two stacks, the bottoms of the two stacks start from two ends of the array and grow towards each other. The first stack Stack1 starts from $m[0]$ and grows in the direction of $m[n-1]$ while the second stack Stack2 starts from $m[n-1]$ and grows in the direction of $m[0]$.

$m[0], m[1], \dots, m[n-2], m[n-1]$

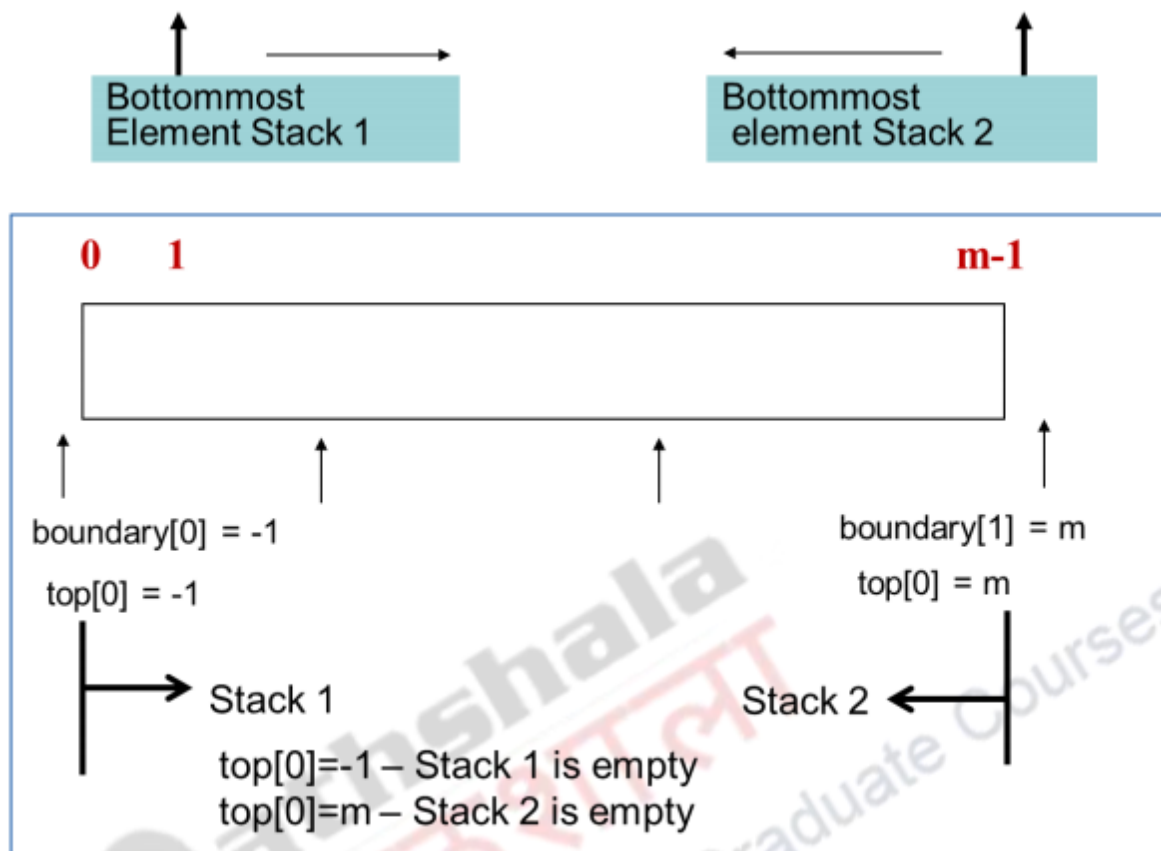


Figure 8.9 An Array Representing Two Stacks

Initially, $\text{boundary}[i] = \text{top}[i]$.

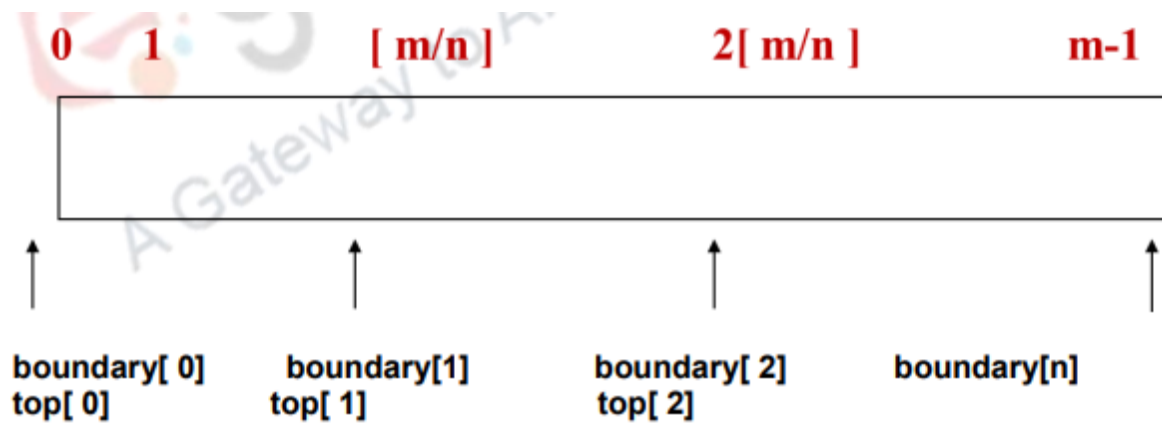


Figure 8.10 Initial configuration for n stacks in memory $[m]$

The same concept can be extended to more than two stacks. Let us assume we want to represent n stacks. The memory is divided into n equal segments (Figure 8.10). We define *boundary* to point to the position immediately to the left of the bottom element while *top* points to the top element.

All stacks are empty and divided into roughly equal segments.
initially

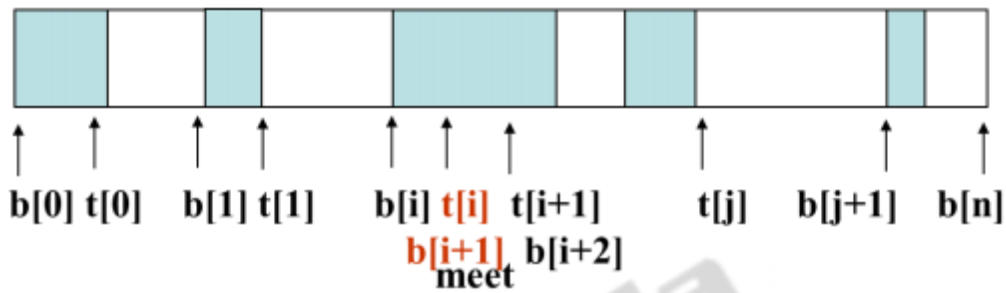
$$\text{boundary}[i] = \text{top}[i] = m / n * i - 1$$

If i -th stack is empty then $\text{top}[i] = \text{boundary}[i]$

If i -th stack is full then $\text{top}[i] = \text{boundary}[i+1]$

Now let us consider a Push operation into one of the n stacks say i th stack. Now a Stack full condition will be signaled when the memory allocated to the i th stack is filled. This condition happens when stack i meets stack $i+1$ which will be signaled when top of i th stack $t[i] = \text{bottom of stack } i+1$ that is $b[i+1]$ (Figure 8.11)

b=boundary, t=top



When stack i meets stack i+1, but the memory is not full

Figure 8.11 Stack i meets Stack i+1

Stack Full Condition

When the stack full condition of i th stack is signaled in this way there may be space available in the array which can be used to grow the segment allocated to the i th stack. In other words, if there is space available (in memory), it should shift the stacks so that space is allocated to the full stack. For this to take place we

- Determine the least, j , $i < j < n$, such that there is free space between stacks j and $j+1$. If there is such a j , then move stacks $i+1, i+2, \dots, j$ one position to the right.
- If there is no such j , then look to the left of stack i . Find the largest j such that $0 \leq j < i$ and there is space between stacks j and $j+1$. If there is such a j , then move stacks $j+1, j+2, \dots, i$ one position to the left.

**Find j , $i < j < n$, $0 \leq j < i$
such that $\text{top}[j] < \text{boundary}[j+1]$**

In the worst case, the function, `stack_full`, has a time complexity of $O(\text{Size of Array})$.

8.6 Linked List Strategy

The Stack ADT can be represented using Linked List or Pointer implementation as shown in Figure 8.12. This essentially means that though the elements of the stack

are logically stored adjacent to each other, they are physically stored in locations that may be far apart but are linked together through the use of pointers.

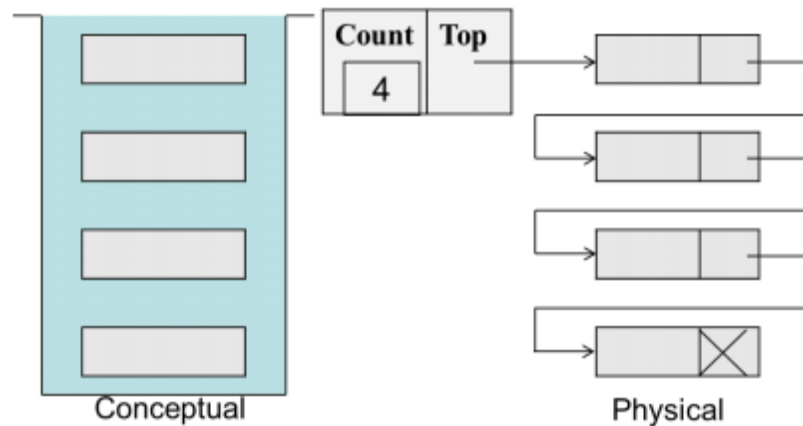


Figure 8.12 Conceptual and Physical Stacks

A Pointer-Based Implementation of the ADT Stack is required when the stack needs to grow and shrink dynamically. In this case Top is a reference to the head of a linked list of items and free nodes need to be supplied during Push operation and to return free nodes during Pop operation. Now let us discuss the linked list implementation of various operations of Stack ADT.

8.6.1 Create Stack

The various steps in the creation of a stack using linked list representation is shown in Figure 8.13. The first step in the creation of stack is the allocation of memory for the Stack head. Here `pnew` is the pointer to the Stack head and the variable `Count` is set to 0 while the variable `Top` is set to Null, Now we return Stack Head `pnew` as the newly created Stack.

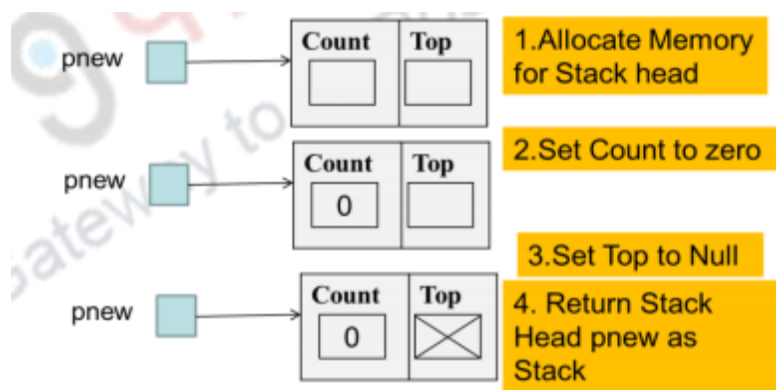


Figure 8.13 Create Operation using Linked List

8.6.2 Push Operation

The push operation is explained with an example (Push(Stack 45)) and illustrated in Figure 8.14. Let us assume that the Stack originally has 4 elements with the **Top** pointing to element **34**. The first step for the Push operation is the allocation of a node **pnew** with two components **Data** and **Link**. The next step is setting the **Data** component of the new node as **45**, the element to be inserted. Now set the **Link** component of the node to the value of **Top**. Now what is left is to change the value of **Top** to the new node **pnew** and to increment **Count** to 5.

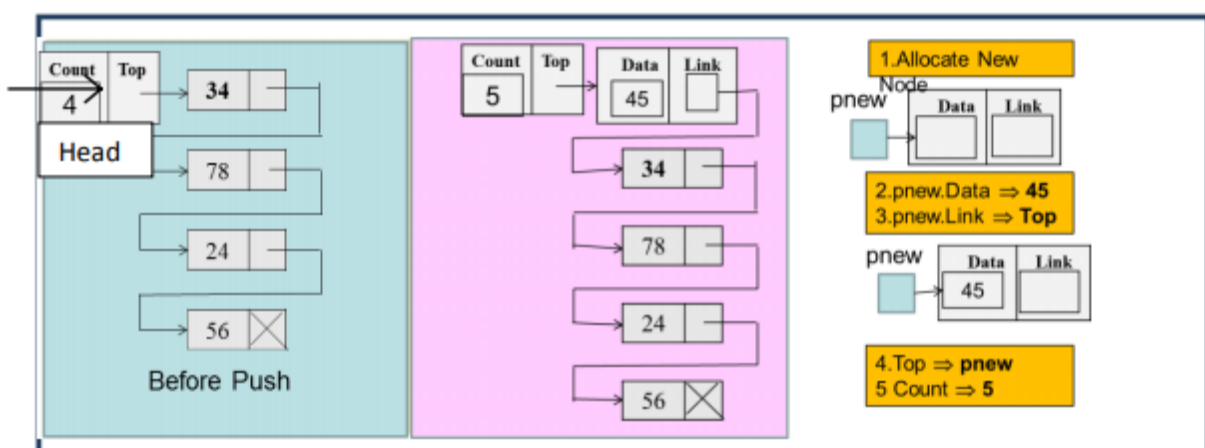


Figure 8.14 Push (Stack, 45) Operation using Linked List

8.6.3 Pop Operation

The pop operation is explained with an example (Pop(Stack)) and illustrated in Figure 8.15. Let us assume that the Stack originally has 4 elements with the **Top** pointing to element **45**. The first step for the Pop operation is checking if the Stack is empty in which case the Pop operation signals a false value indicating that the operation is not possible. Now we set **Dataout** to the **Data** component of the **Top**.

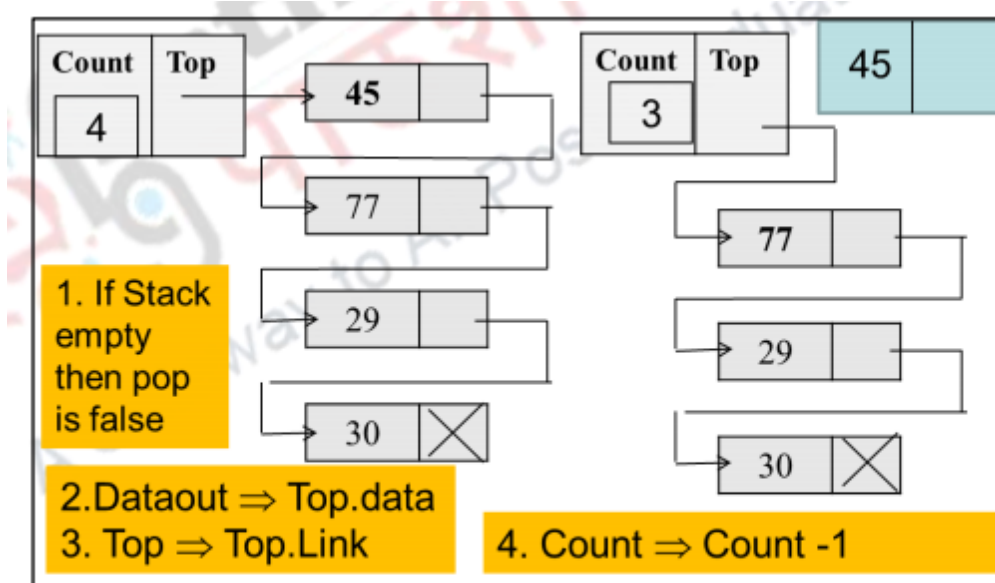


Figure 8.15 Pop (Stack) Operation using Linked List

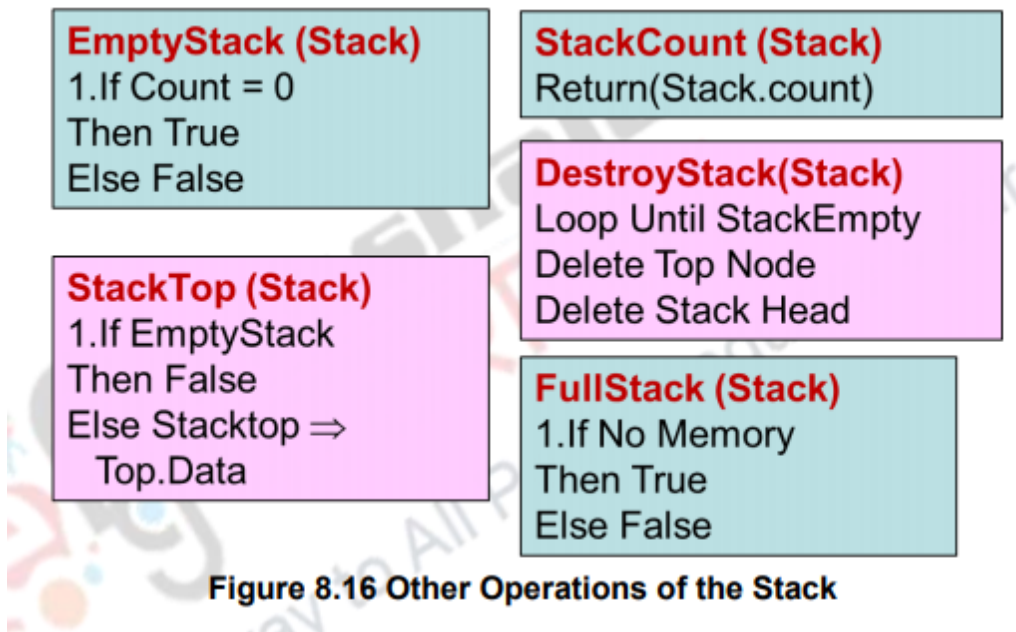
Now we modify Top to point to the element that **Top** pointed to (**Top.Link**) The next and final step is decrementing the value of **Count** to 3.

8.6.4 Other Operations

There are other operations (Figure 8.16) that are associated with the Stack. These operations could be

- Checking whether the Stack is empty where we need to just check whether Count is 0.
- Retrieving the top element of the stack where we need to check whether the Stack is empty else StackTop is Top.Data.
- StackCount just returns Stack.Count.

- DestroyStack will Pop Top Node until Stack is empty and then we delete Stack Head.
- Finally we can check if the stack is full – which will be signalled when we run out of memory although this operation is not normally specified for Linked List implementation of Stack ADT.



8.7 Using List ADT

Another important implementation of the Stack ADT is using another ADT – the List ADT. This implementation is also important since this is an example of the usage of the List ADT. Now if we assume that the item in position 1 is the top of the Stack. In this case Figure 8.18. Here instead of assuming that stack grows from 1 to n we assume that it grows towards n. Therefore the adtlist used here is doing insertions and deletions at the front only.

```

List aList; // list of stack items
isEmpty(Stack)
return aList.isEmpty();
push(Stack,newItem)
return aList.insert(1,newItem)

```

```
pop(Stack)
if (aList.retrieve(1,stackTop) return aList.remove(1); Else return false;
```

Figure 8.18 (a) Stack ADT using List ADT



Figure 8.18 (b) Stack ADT using List ADT

8.8 Comparing Implementations

An array-based implementation needs the size of the stack to be fixed and therefore the push operation cannot add an item to the stack if the stack's size limit has been reached. On the hand, the size of the stack is dynamic in a pointer-based implementation where we do not put a limit on the size of the stack. Another implementation is the use of the ADT list implemented say for example using pointers. The ADT list approach reuses an already implemented class. This is much simpler to write and saves time.

Summary

- Explained the three different implementations of Stack ADT
- Discussed array implementation of stack
- Explained the linked list implementation of stacks
- Outlined the ADT list based implementation of stacks

you can view video on Implementation of Stack ADT



Web Links
sce.uhcl.edu/shresthab/csci3033/Lecture6.ppt
www.csie.kuas.edu.tw/course/CS/old/english/ch-12.ppt
www.cs.nyu.edu/~melamed/courses/102/lectures/Stacks.ppt
www.cs.sfu.ca/CourseCentral/225/jmanuch/lec/4-3.ppt
www.cs.bilkent.edu.tr/~adayanik/courses/cs201/slides/stacks.ppt
http://www.ece.uwaterloo.ca/~ece250/Algorithms/Array_resizing/
monet.skku.ac.kr/course_materials/graduate/ds/lecture/datastruct03.pdf
www.cse.unsw.edu.au/~cs2011/lect/05_Stacks4b.pdf
http://www.icodeguru.com/vc/10book/books/book1/chap03.htm
Supporting & Reference Materials
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Adam Drozdek, "Data Structures and Algorithms in C++", Cengage; 4th edition, 2013
Alfred V. Aho and Jeffrey D. Ullman, "Data Structures and Algorithms", 1983
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014